

Network Administration / System Administration

Homework 11

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

Discussion Partners:

- Classmates : 賴泓安、鄭竣陽、喻慈恩

1 三角準則的侵略者!?

2 果汁店也有洞!

Reference:

- Lab 13 Slide
- [Understanding SSRF, XSS, and CSRF: The Triple Threat in Web Security](#)
-
-

p.s. 為了完成這題，我先按照 Lab 13 的指示，開啟了一個 docker container，來 host Juice Shop 的網站，並且前往 <http://localhost:3000> 來進行操作。

2. (a) Juice Shop hacking challenge

DOM XSS ★

Steps:

1. 我在搜尋欄裡面輸入 `<iframe src="javascript:alert(`xss`)">`，即完成這題。

Confidential Document ★

Steps:

1. 我前往 “About Us” 頁面並點擊 “Check out our boring terms of use if you are interested in such lame stuff” 連結。
2. 透過該連結會前往 <http://localhost:3000/ftp/legal.md>。根據網址，我們可以發現 <http://localhost:3000/ftp/> 是一個 ftp 目錄，裡面有許多檔案，而且大部分的檔案都可以被下載。
3. 我點擊 “acquisitions.md” 檔案，並且在瀏覽器中查看其內容。發現他是所謂的 “confidential document”，即完成這題。

View Basket ★★

Steps:

1. 我以隨便一個帳號登入 Juice Shop，並開啟 Burp 的 Proxy，啟動 Intercept。接著，我在 Juice Shop 的首頁點擊 “Your Basket” 按鈕。
2. 根據攔截到的請求，我們可以看到一個 GET 請求，URL 為 `/rest/user/whoami`，然後又看到一個 POST 請求，URL 為 `/rest/basket`。

16:03:17 24 ...	WS	← To client		http://localhost:3000/socket.io/?EIO=4&transport=websocket&sid=mwY7pyjzibfqCnPEAAAC
16:03:17 24 ...	HTTP	→ Request	GET	http://localhost:3000/rest/basket/2
16:03:17 24 ...	HTTP	→ Request	GET	http://localhost:3000/rest/user/whoami
16:03:24 24 ...	HTTP	→ Request	GET	http://localhost:3000/socket.io/?EIO=4&transport=polling&t=PS1KN97

Figure 1: 攔截到的請求

3. 在 `/rest/basket` 的請求中，我們可以看到一個名為 `userId` 的參數，這個參數的值是我們登入的帳號的 ID。我將它改為 1，然後 forward 該請求。
4. 關閉 Intercept 後，就發現我們進入了另一個帳號的購物籃頁面，並且可以看到該帳號的購物籃內容。完成此題。

Five-Star Feedback ★★

Steps:

1. 根據提示，我需要先前往 admin session。
2. 我在網址列輸入 `http://localhost:3000/#/administration`，成功進入管理員頁面。(順便解了另一題：Admin Session)
3. 在管理員頁面中，我在“Customer Feedback”區塊中，將看的到的五星評價都刪除，即完成此題。

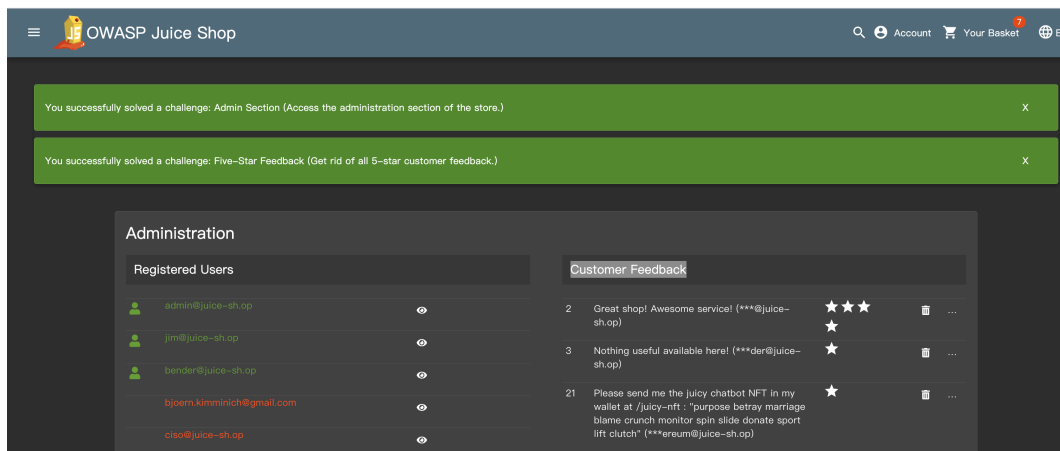


Figure 2: 管理員頁面中的 Customer Feedback 區塊

Exposed credentials ★★

Steps:

1. 根據提示，我按 F12 開啟開發者工具，並切換到 Sources 分頁，並開啟 `main.js` 檔案。
2. 我搜尋 `testing`，發現有一組帳號密碼 `testingUsername` 和 `testingPassword`，其值分別為 `testing@juice-sh.op` 和 `IamUsedForTesting`。
3. 使用這組帳號密碼登入 Juice Shop，即完成此題。

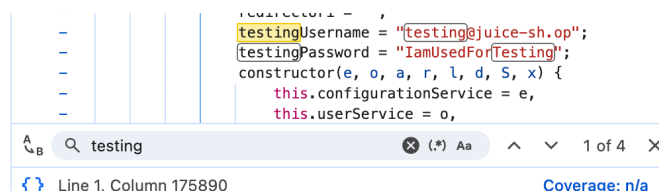


Figure 3: 在 main.js 中找到的帳號密碼

Scoreboard 截圖：

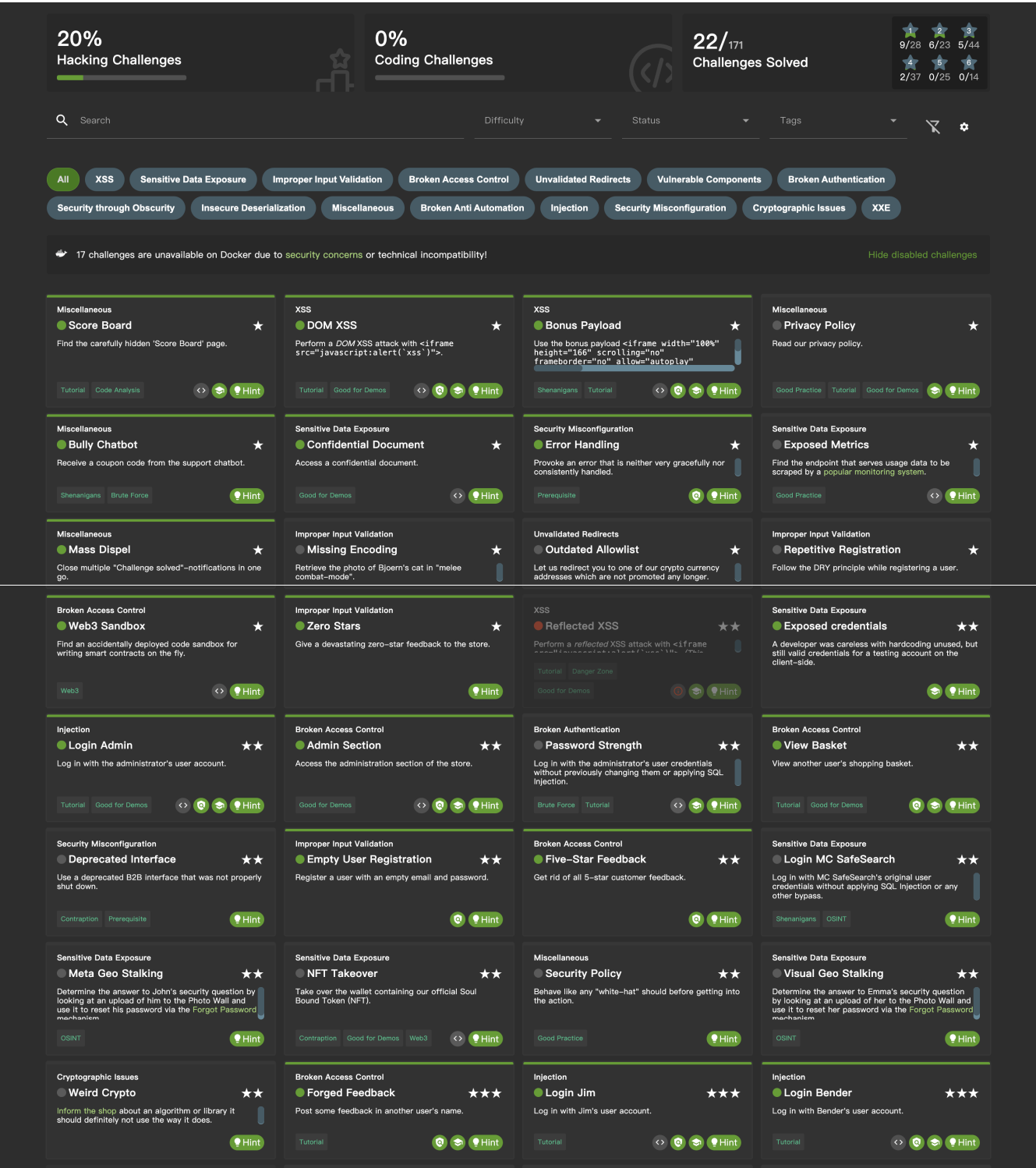


Figure 4: Juice Shop 的 Scoreboard

2. (b) SSRF、CSRF 和 XSS

- SSRF (Server-Side Request Forgery) :

在這種攻擊方式中，攻擊者利用伺服器端的向特定的另一個服務發送請求，通常是為了繞過防火牆或存取內部網路資源。常見的情況是，假如你的網站上有個輸入框，讓使用者自己填網址。攻擊者就可以偷偷填一個公司內網的地址，讓你的伺服器去「自動」幫他連過去。結果可能拿到內網機密、撈到不該給外部看的資料，甚至繞過防火牆去搞其他事。

- CSRF (Cross-Site Request Forgery) :

在這種攻擊中，攻擊者誘使使用者在已經登入的情況下，透過攻擊者先在別的網站或郵件裡夾一個隱藏的請求連結，發送未經授權的請求到另一個網站。只要你正好在同一瀏覽器裡已經登入了某個網站，瀏覽器就會帶上你的 Cookie / 驗證資訊，自動幫你送出那個請求，並用你的身份執行操作。結果你可能就被偷轉帳、改設定或亂發文。

- XSS (Cross-Site Scripting) :

基本上，攻擊者把惡意 JavaScript 或 HTML 混進你信任的網頁裡，等人一打開網頁，腳本就在使用者的瀏覽器跑起來。可以偷 Cookie、竊取表單資料，甚至直接在頁面上做釣魚操作。

- 三者比較:

攻擊類型	SSRF	CSRF	XSS
攻擊對象	伺服器	使用者	使用者
攻擊方式	利用伺服器發送請求	誘使使用者發送請求	注入惡意腳本
影響範圍	伺服器內部資源	使用者帳號	使用者瀏覽器

Table 1: SSRF、CSRF 和 XSS 的比較

3 R-SA ! 破密部

4 TESTING in the FUZZ

5 敗北協定太多了！

6 猫物語（赤）

Reference:

- [Linear congruential generator \(LCG\) - Wikipedia](#)
- [LCG 隨機數破解](#)
- [Modular multiplicative inverse function in Python - Stack Overflow](#)
- <https://blog.csdn.net/PlutoZuo/article/details/134697641>
-
-
-
-
-

6. (a) 猜數字

Steps:

1. 首先，我觀察了 `fatcat.py` 的程式碼，發現它使用了一個線性同餘生成器（LCG）來產生隨機數。這個生成器的公式為：

$$X_{n+1} = (a \cdot X_n + c) \pmod{m}$$

2. 在程式碼當中我們已知 m ，但是並不知道 a 和 c 的值。然而，根據我搜尋的結果，LCG 演算法的 a 和 c 的值可以透過測量三次隨機數的差值來計算出來。過程如下：

$$\begin{aligned} X_1 &\equiv a X_0 + c \pmod{m}, \\ X_2 &\equiv a X_1 + c \pmod{m}, \\ X_2 - X_1 &\equiv a(X_1 - X_0) \pmod{m} \end{aligned} \quad \Rightarrow \quad \begin{aligned} a &\equiv \frac{X_2 - X_1}{X_1 - X_0} \pmod{m}, \\ c &\equiv X_1 - a X_0 \pmod{m}. \end{aligned}$$

3. 我根據和伺服器互動三次，取得了 X_0 、 X_1 和 X_2 的值。接著，我透過 `solve_lcg.py` 程式碼計算出 a 和 c 的值。
4. 我撰寫了 `predict_lcg.py` 程式碼，並且在裡面透過 `pwntools` 套件來和伺服器互動，並且使用計算出來的 a 和 c 的值來預測下一個隨機數。猜中答案 100 次後，我取得足夠多信任值，並且成功獲得了 flag。

Flag:

`NASA_HW11{pseudorandomness_does_not_guarantee_unpredictability}`

6. (b) 考試答案卷

Steps:

1. 觀察程式，可以看到產生 flag 2 加密的程式如下：

```

1 def OTPEncrypt(msg: bytes) -> bytes:
2     key_len = 10
3     key = secrets.token_bytes(key_len)
4     enc = bytes(msg[i] ^ key[i % key_len] for i in range(len(msg)))
5     return enc
6
7 def send_exam_result():
8     hexlify(OTPEncrypt(FLAG2.encode())).decode()

```

這個加密方式將一組長度為 10 的 bytes 陣列，去和密鑰做 XOR 運算。因為我們知道原本文字當中一定含有 NASA_HW11{ 開頭的字串，且密文 XOR 明文會得到密鑰，所以我們可以透過這個已知的字串來破解密鑰。

2. 在我的 decrept_otp.py 當中，我首先先用 unhexlify 將密文轉換成 bytes，然後再窮舉 flag 開始的位置，接下來找出每一個開始位置如果明文是 NASA_HW11{ 時，key 要是多少。
3. 接著，我在嘗試用這些 key 去解密密文，直到找到一個，解密出的文字包含整個 flag 字串的 regex 的 key。雖然有兩個開始位置算出的 key 都符合 regex，但是其中一個 key 解密出的文字明顯是亂碼，而另一個則是正常的文字。
4. 我解密出的文字是：

```

1 LinearAlgebraMidterm Name:Fatcat Score:86
  ↪ Flag:NASA_HW11{07p_k3y_mu57_b3_47_13457_45_10n6_45_pl41n73x7}

```

因此可以得到 flag。

Flag:

NASA_HW11{07p_k3y_mu57_b3_47_13457_45_10n6_45_pl41n73x7}

6. (c) PoW

6. (d) Shared Key